FIAP / CIDADES ESG INTELIGENTES

HydroGuard

CI/CD, containerização e orquestração de ambientes

API de monitoramento de consumo de água com indicadores ESG, cadastro de medidores, relatórios e alertas de possíveis vazamentos.

Integrantes

Preencher antes da entrega:
Nome completo | RM
Ainda não informado

Objetivo da atividade

Automatizar build, testes e deploy; empacotar a aplicação em containers; orquestrar API e banco; documentar arquitetura e evidências de staging e produção.

Situação: implementação preparada; execução externa pendente

Os testes estão concentrados em C# e foram executadas somente verificações estáticas nesta revisão. O ambiente de preparação não tem .NET nem Docker. Não houve execução do GitHub Actions, publicação de imagem ou deploy externo. Os prints obrigatórios e os nomes/RMs precisam ser acrescentados antes da entrega final.

Escopo ESG

Ambiental: apoiar identificação de consumo elevado e desperdício de água. Social: facilitar acesso a informações por uma interface web. Governança: autenticação, configuração externa de segredos e rastreabilidade por commit e digest. Não foram medidos ganhos ambientais ou validados indicadores como certificação ESG.

01 Pipeline CI/CD

GitHub Actions coordena o fluxo. GitHub Container Registry armazena a imagem. Os jobs de CI usam Ubuntu hospedado pelo GitHub; deploys usam runners Linux x64 persistentes nos hosts dos ambientes.

Etapa	Lógica e condição de avanço
1. Build e testes	Restore, build Release, 20 testes C# com xUnit. Publica TRX/cobertura. Falha bloqueia imagem e deploy.
2. Container e smoke	Constrói imagem e sobe Compose com SQL real. Valida readiness, frontend, Swagger, autenticação e rotas medidores/ESG.
3. Publicação GHCR	Somente main: publica a imagem validada com tag do commit e obtém digest SHA-256.
4. Staging	Pull por digest, Compose persistente e smoke validando Staging + revisão. Produção depende do sucesso.
5. Produção	Promove o MESMO digest sem recompilar. Smoke valida Production + revisão.

Gatilhos e política

Pull requests: validação de CI sem publicação/deploy. Push em main ou execução manual em main: fluxo completo. Uma execução por branch evita deploys simultâneos. Produção avança automaticamente após staging; revisores obrigatórios podem ser adicionados se a política mudar.

Separar CI de ambientes persistentes

A stack hydroguard-ci é temporária e removida ao fim do job. Ela não é evidência de staging. Os projetos hydroguard-staging e hydroguard-production continuam em execução nos hosts após o job.

Artifacts gerados após execução real

test-results: TRX e cobertura. container-evidence: smoke e digest. staging-evidence e production-evidence: smoke, estado dos containers e revisão. Nesta versão esses artifacts externos ainda não existem.

02 Docker e arquitetura

Dois serviços por ambiente: API ASP.NET Core, que também serve o frontend, e SQL Server 2022 Express. A aplicação escuta 8080 dentro do container; a porta do host distingue os ambientes.

Recurso	Configuração
Imagem da aplicação	Multi-stage: SDK 8 para restore/publish; ASP.NET 8 bookworm-slim para runtime. Inclui curl para health check; executa como usuário app.
Banco	SQL Server 2022 Express, acessível somente pela rede interna Docker; porta 1433 não publicada no host.
Persistência	Volume sqlserver_data com prefixo do projeto Compose; dados sobrevivem à recriação de containers.
Redes	frontend para a API e database interna compartilhada por API/banco; redes diferentes por projeto.
Inicialização	SQL passa health check; API aplica migrations e cria admin inicial. API saudável exige conexão ao banco.
Variáveis	ConnectionStrings, Jwt e Bootstrap recebidas em runtime. Secrets separados por ambiente; não entram na imagem.

Ambientes isolados

Ambiente	Projeto Compose / porta / seed
Local	hydroguard-local / 5000 / dados de demonstração habilitados
Staging	hydroguard-staging / 5001 / somente usuário admin
Produção	hydroguard-production / 5000 / somente usuário admin

Identificação da imagem

```
Tag: ghcr.io/<proprietario>/<repositorio>:<commit>
Deploy: ghcr.io/<proprietario>/<repositorio>@sha256:<digest>
```

O digest real será conhecido após publicação. /healthz retorna ambiente e revisão do commit; /livez indica apenas que o processo responde. Imagens-base e SQL usam tags móveis nesta versão. Fixar seus digests é uma evolução para reprodutibilidade integral.

03 Execução e preparação dos hosts

Execução local

```
python3 scripts/setup_env.py
docker compose -p hydroguard-local config --quiet
docker compose -p hydroguard-local up -d --build \
  --wait --wait-timeout 300
docker compose -p hydroguard-local ps
```

Acesse http://localhost:5000 e /swagger. Entre como admin usando ADMIN_PASSWORD do .env gerado. No Windows, use python quando python3 não estiver disponível. Para parar preservando dados: docker compose -p hydroguard-local down.

Testes de integração

```
dotnet restore HydroGuard.sln
dotnet build HydroGuard.sln -c Release --no-restore
dotnet test tests/HydroGuard.Tests/HydroGuard.Tests.csproj -c Release --no-build
```

SDK .NET 8 é necessário para xUnit e smoke C#; o pipeline prepara o SDK nos jobs. Há 20 testes xUnit e um executável smoke C# sem pacotes externos. InMemory não prova comportamento relacional: a etapa container usa SQL Server real. Python foi removido dos testes e do pipeline; permanece apenas nos geradores opcionais de .env/PDF.

Preparação do GitHub e ambientes

1. Publique o projeto em repositório GitHub com main e Environments disponíveis. Crie staging e production e limite deployments à main.

2. Nos hosts Linux x64, instale Docker, Compose v2.24+, Bash e Git; o workflow prepara o SDK .NET 8. Registre runners usando os comandos de Settings > Actions > Runners.

3. Atribua labels hydroguard-staging e hydroguard-production. Instale runners como serviços. Em um host único, use dois processos/pastas e recursos para duas stacks.

4. Em cada environment, configure secrets MSSQL_SA_PASSWORD, JWT_KEY e ADMIN_PASSWORD. Use valores diferentes. Configure variables API_PORT, BIND_ADDRESS e APP_URL.

5. Faça push em main, acompanhe os quatro jobs e acesse ambos os ambientes. APP_URL cria apenas um link; domínio, proxy HTTPS ou túnel devem ser preparados no host.

O README detalha cada passo e acesso ao GHCR por GITHUB_TOKEN. Senhas do SQL/admin devem ter 12+ caracteres; JWT deve ter 32+ bytes. BIND_ADDRESS padrão é 127.0.0.1.

04 Evidências verificadas nesta preparação

Verificações estáticas da revisão com testes C#

O registro abaixo foi produzido por verificações reais de arquivos, sintaxe Bash, YAML e referências de projetos. Como não há SDK .NET ou Docker aqui, nenhum teste C# nem smoke HTTP foi executado. Esse registro não comprova funcionamento da API ou dos deploys.

```
HydroGuard - revisão com testes exclusivamente em C#
Data UTC: 2026-10-13T02:00:29.455612+00:00
Escopo: verificações estáticas; não comprova build, testes ou deploy.

$ bash -n scripts/deploy.sh
Exit code: 0
YAML parse e sintaxe Bash dos passos inline: OK
Jobs, dependências, instalação SDK e promoção do mesmo digest GHCR: OK
Sem Python no workflow, deploy ou diretório de testes: OK
3 projetos .NET 8 na solution; XML e referências de projetos: OK
20 testes xUnit declarados (não executados): OK
Smoke HTTP e pré-validação em C#, sem pacotes externos: OK
Compose: SQL sem porta pública, volume e rede interna: OK

Execuções pendentes:
dotnet: não instalado neste ambiente
docker: não instalado neste ambiente
Build C# e testes xUnit/smoke: NÃO executados nesta preparação.
GitHub Actions, imagem GHCR e deploys staging/produção: NÃO executados.
Nomes/RMs e prints reais: ainda pendentes.
```

Arquivo completo: evidence/validacao-local.txt. O ZIP inclui esse registro. Nenhum resultado .NET/Docker foi inferido a partir dessas verificações.

05 Prints obrigatórios: captura pendente

Pipeline com build, testes e deploy

PENDENTE - execução GitHub Actions

Capturar run real com build-test, container, deploy-staging e deploy-production aprovados. Incluir logs de build/testes, resultado xUnit, commit e digest da imagem. Registrar URL da run em docs/evidencias.md.

Staging funcionando

PENDENTE - ambiente staging

Capturar interface acessível no host de staging, containers saudáveis e /healthz retornando Staging + revisão do commit. Baixar staging-evidence. O nome do GitHub Environment não comprova funcionamento.

Produção funcionando

PENDENTE - ambiente de produção

Capturar interface de produção, containers saudáveis e /healthz retornando Production + a mesma revisão. Confirmar que o digest da API coincide com staging; baixar production-evidence.

Como incorporar evidências ao documento

Adicione imagens reais em docs/prints/pipeline.png, staging.png e production.png. Atualize nomes/RMs em docs/integrantes.txt e links em docs/evidencias.md. Execute pip install -r docs/requirements.txt e python3 docs/generate_pdf.py. O gerador inclui as imagens disponíveis antes do checklist final. Atualize os textos e o checklist para refletir os resultados reais; acrescentar uma imagem não muda automaticamente o status.

Não exponha .env, senhas, tokens ou secrets nos prints. Use config --quiet e compose ps. Teste a URL pública separadamente quando houver proxy: o smoke automatizado usa a porta local do host.

06 Desafios e decisões técnicas

Desafio	Solução e limite
Ausência de SDK/Docker	Pipeline preparado para executar em runners com ferramentas. Execução completa continua pendente; evidências não foram fabricadas.
Segredos fixos	Removidos do código/configuração. Geração aleatória local e secrets de environment. Admin inicial criado por Bootstrap.
Testes com providers conflitantes	SQL não registrado em Testing; factory registra InMemory com nome único. Importação de Hosting acrescentada para as extensões de teste. Validação xUnit pendente.
Ordem de inicialização	Health check SQL com sqlcmd, depends_on service_healthy e readiness API com conexão real.
Colisão entre ambientes	Sem nomes fixos de container; projetos, portas, volumes e redes separados.
Rastreabilidade de releases	GHCR mantido. Commit na imagem e deploy por digest; smoke C# confirma ambiente e revisão.

Limites operacionais

Uma réplica da API por stack, migrations no startup e possibilidade de breve indisponibilidade ao atualizar. Sem rollback automático, backups automáticos ou alta disponibilidade. Produção com dados reais exige evolução da gestão de permissões SQL, TLS, observabilidade e recuperação.

Rollback e persistência

Reaplique o digest anterior usando scripts/deploy.sh e a revisão correspondente, com os mesmos secrets do ambiente. Reverter imagem não reverte schema. Não use down --volumes em staging/produção. Mudanças de secret não trocam automaticamente senhas já armazenadas no banco.

Referências oficiais

GitHub Actions: deploy: documentação oficial

Docker Compose: ordem de inicialização: documentação oficial

Microsoft: SQL Server em containers: documentação oficial

07 Checklist de entrega

Preenchido de acordo com o que está comprovado nesta versão. Arquivo implementado não significa execução aprovada. Atualize após os deploys reais.

Item obrigatório	Situação nesta versão
Projeto ZIP organizado e completo	PREPARADO - inclui código, configs, scripts e documentação.
Dockerfile funcional	IMPLEMENTADO - execução funcional ainda não comprovada.
Docker Compose / Kubernetes	COMPOSE IMPLEMENTADO - volume, variáveis e redes; execução pendente.
Pipeline build, teste e deploy	CONFIGURADO - GitHub Actions com dois deploys; run externa pendente.
README com instruções e prints	INSTRUÇÕES PRONTAS - prints/links externos pendentes.
PDF/PPT técnico com evidências	PDF PREPARADO - inclui log local; faltam prints externos obrigatórios.
Deploy realizado em staging	PENDENTE - requer host/runner, secrets e execução real.
Deploy realizado em produção	PENDENTE - depende de staging aprovado e execução real.
Integrantes e RMs	PENDENTE - nomes completos e RMs não informados.

Verificações adicionais

[OK] Testes Python removidos; smoke migrado para C#; sintaxe Bash e análise YAML verificadas.
[PENDENTE] Build .NET, 20 testes xUnit, build Docker e validação Compose real.
[PENDENTE] Imagem GHCR publicada e mesma referência usada nos dois ambientes.
[PENDENTE] Artifacts e prints reais adicionados ao README/PDF.

A entrega final ainda depende das execuções externas

Este documento registra uma implementação preparada. Para cumprir integralmente a atividade, execute o pipeline e os dois deploys, acrescente as evidências reais e atualize os itens pendentes.